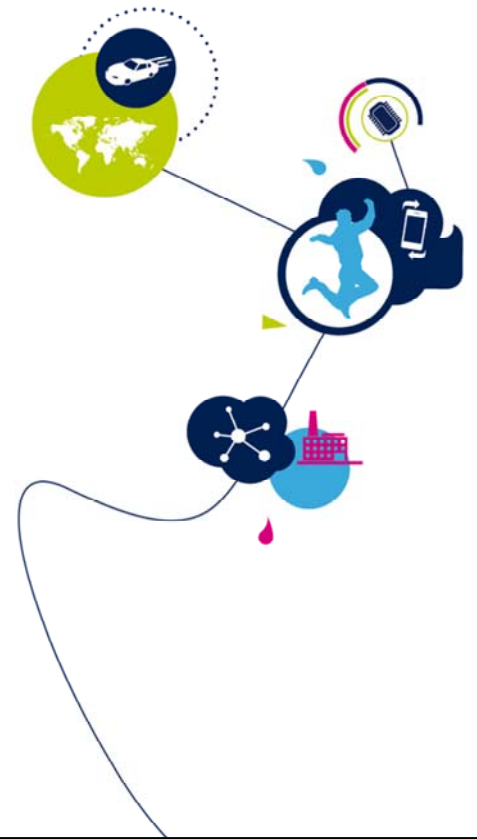


STM32MP1 - DBG

Debug and trace
Revision 1.0



Hello, and welcome to this presentation of the STM32 debug and trace interface. It covers the debug and trace capabilities offered by STM32MP1 devices.

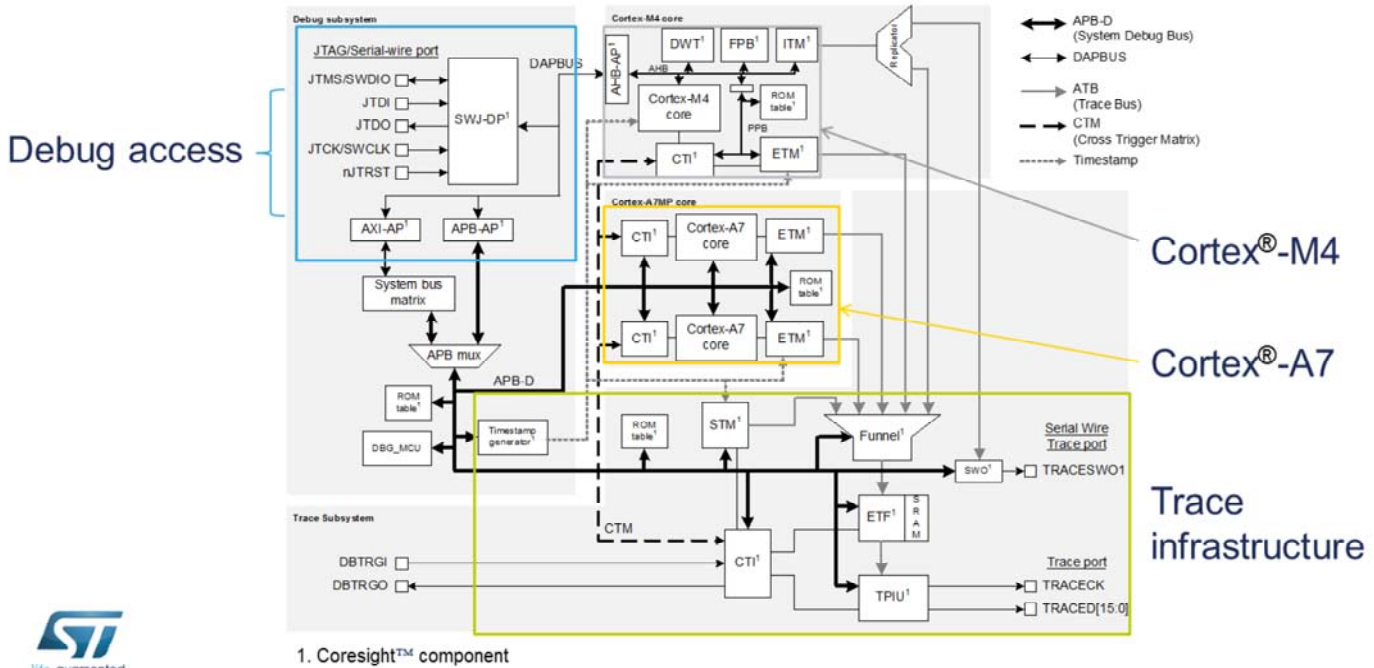


- STM32MP15 provides rich support for debug and trace
 - Download programs into RAM or Flash memory
 - Examine memory and register contents
 - Insert breakpoints and halt the processor
 - Run or Single-step through programs
 - Trace program execution
- Based on ARM® CoreSight™ architecture
 - Wide range of compatible tools
 - Standard interface (JTAG/serial wire)

The STM32MP1 microprocessor incorporates all the familiar debug capabilities provided by the STM32 family of MCUs – flash download, breakpoint debugging, register and memory view, serial wire trace – and adds high bandwidth instruction trace as well as cross-triggering capabilities in multi-core versions of the STM32MP1 family. The debug and trace infrastructure uses the ARM® CoreSight™ standard, well supported by most tool providers.

Debug architecture

3



The debug and trace infrastructure is composed of four distinct functional domains:

- Debug access infrastructure – includes the debug port (SWJ-DP) and access ports (AP) which allow access by an external debugger to the target's trace and debug features.
- Trace infrastructure – includes the serial (SWO) and parallel (TPIU) trace ports, the trace FIFO (ETF) used for smoothing the trace flow, and the trace funnels which combine the trace from each source into a single flow. There is also a System Trace Module (STM) which allows software generated debug information, as well as hardware events, to be traced.
- Cortex-A7 core – includes the processor (single- or dual-core) and embedded trace module (ETM).
- Cortex-M4 core – includes the processor and associated debug and trace units (DWT, FPB, ITM, ETM)

In addition, there are system debug features including:

- Cross trigger interfaces and matrix (CTI, CTM) – these allow simultaneous halting of both cores, triggering of trace, etc.

- Global timestamp generator – provides a common time reference for the different trace sources
- DBG_MCU – provides proprietary features such as freezing of timers during debug
- External trigger input/output – allows an external signal to trigger debug or trace, or generates a trigger pulse for synchronizing external equipment or components

- The debugger accesses the STM32MP15 via the JTAG/SWD debug port
 - Standard 5-pin JTAG port also used for boundary scan and DFT
 - Serial wire debug (SWD) port uses 2 of the JTAG port pins

Pin	JTAG debug port	SW debug port
JTMS-SWDIO	Test Mode Select	Serial Wire Data (In/Out)
JTCK-SWCLK	Test Clock	Serial Wire Clock
JTDO	Test Data Out	-
JTDI	Test Data In	-
nJTRST	Test Reset	-



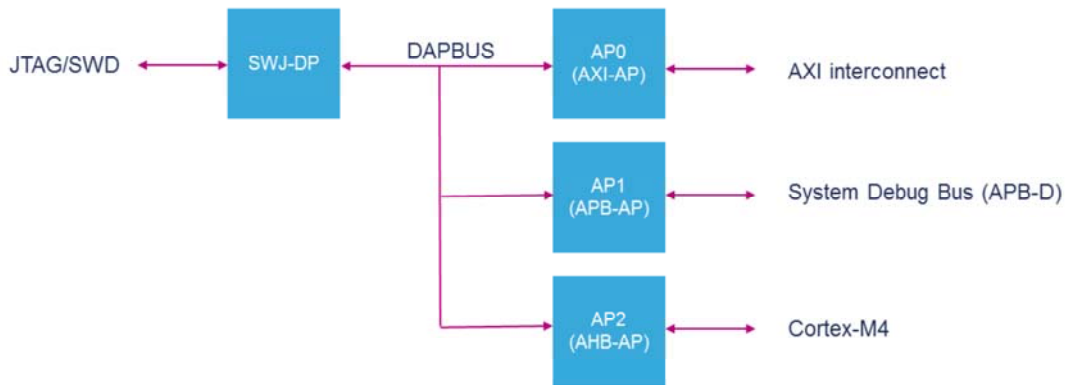
The debug port is available on dedicated pins on all STM32MP1 packages.

Serial wire debug uses a special serial code driven by the debugger on the SWDIO (JTMS) input. This is recognized by the SWJ-DP which switches to SWD mode (after reset JTAG mode is configured by default).

ST-Link, and most 3rd party debug adaptors (for example, Ulink), support serial wire debug.

Access ports 5

- Three access ports (AP) act as bus masters allowing the debugger to perform read/write transactions to memory and registers



AP0: Allows access to the AXI interconnect. This gives the debugger direct access to all memory and peripheral registers.

AP1: Allows access to the debug and trace features on the system APB debug bus, namely the Cortex-A7 debug features and the trace subsystem.

AP2: Allows access to the debug and trace features integrated in the Cortex-M4 processor core via its internal AHB bus.

Application running on either processor can access the debug features located on the System Debug Bus since they are mapped in the unified address space. This includes the trace subsystem (STM, TPIU, TSGEN and ETF), as well as the Cortex-A7 features (ETM, CTI, DBG). However only the Cortex-M4 can access features on its

private bus.

Authentication and security

6

- 6 authentication signals from the Boot and Security (BSEC) unit control the access to debug features:
 - **DEVICEEN**: Controls access to debug components via the external debug port.
 - 0: External debugger has no access to debug components. Debugger access to system interconnect is not affected.
 - 1: External debugger has access to debug components
 - **DBGEN**: Global enable for all debug and trace features
 - 0: All debug features are disabled. External and software access to debug components is still possible. External access to system interconnect is disabled.
 - 1: Debug features in non-secure mode are enabled. External access to non-secure system interconnect is enabled. Debug features in secure mode and external access to secure system interconnect are dependent on the state of the SPIDEN signal.
 - **SPIDEN**: Enables debug in secure privileged mode when DBGEN = 1
 - 0: Debug features are disabled in secure privileged mode. External access to secure system interconnect is disabled.
 - 1: Debug features are enabled in secure privileged mode. External access to secure system interconnect is enabled.
 - **NIDEN**: Enables trace and performance monitoring (non-invasive debug)
 - 0: Trace generation is disabled.
 - 1: Trace generation in non-secure mode is enabled. Trace generation in secure mode is dependent on the state of the SPNIDEN signal.
 - **SPNIDEN**: Enables trace and performance monitoring in secure privileged mode when NIDEN = 1.
 - 0: Trace generation is disabled in secure privileged mode.
 - 1: Trace generation is enabled in secure privileged mode.
 - **DBGSWEN**: Enables self-hosted debug
 - 0: Software access to all debug components is disabled
 - 1: Software access to all debug components is enabled

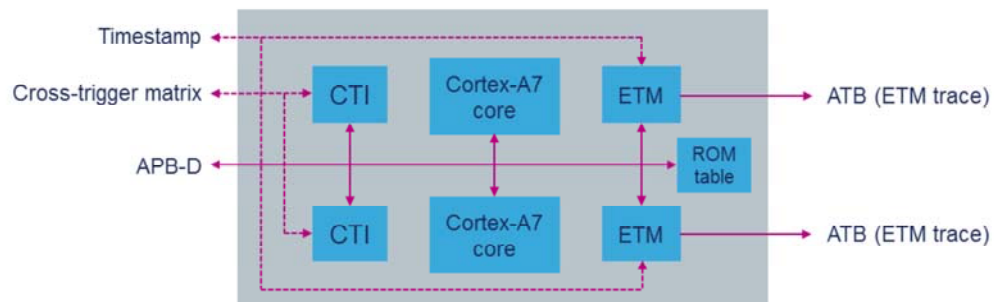


The authentication signal states are set in the Boot and Security (BSEC) unit. The default state of these signals are determined by the factory state of the device (open or closed). The state can be modified by a secure software.

The debugger must use secure privileged transactions to access secure addresses. These transactions only succeed if the SPIDEN signal is asserted.

Cortex[®]-A7 T&D features 7

- The Cortex-A7 contains the following debug components:
 - One debug unit (DBG) per core
 - One Embedded Trace Macrocell (ETM) per core
 - One Cross-Trigger Interface (CTI) per core
 - ROM table



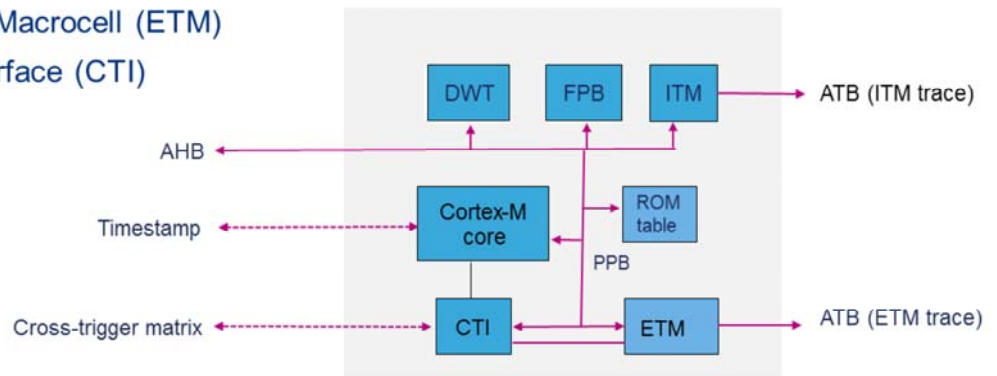
All debug related registers in the Cortex-A7 core are accessed via the system debug bus, APB-D, through access port AP1. The ROM table contains pointers to the base addresses of each debug component in the core. ROM tables are used by some debug tools to automatically detect the topology of the CoreSight™ infrastructure in the target. The debug unit (DBG) contains the registers for controlling the processor core while in debug mode.

Cortex[®]-M4 T&D features

8

- The Cortex[®]-M4 contains the following debug components:

- System Control Space (SCS)
- Data Watchpoint and Trace Unit (DWT)
- Breakpoint Unit (FPB)
- Instrumentation Trace Macrocell (ITM)
- Embedded Trace Macrocell (ETM)
- Cross-Trigger Interface (CTI)
- ROM table



All debug related registers in the Cortex-M4 core are accessed via the dedicated AHB access port AP0.

The ROM table contains pointers to the base addresses of each debug component visible from the AP. ROM tables are used by some debug tools to automatically detect the topology of the CoreSight[™] infrastructure in the target.

The SCS (system control space) contains the registers for controlling the processor core while in debug mode.

The other units are described in the following slides.

Data Watchpoint and Trace Unit

9

- The DWT provides four comparators that can be used as:
 - Watch point
 - ETM trigger
 - PC sampling trigger
 - Data address sampling trigger
 - Data comparator
 - Clock cycle counter comparator
- It also contains counters for software profiling:
 - Clock cycles
 - Folded instructions
 - Load Store Unit (LSU) operations
 - Sleep cycles
 - Number of cycles per instruction
 - Interrupt overhead



A Data Watchpoint (DWT) comparator compares one of the following with the value held in its DWT_COMP register:

- A data address
- An instruction address
- A data value
- The cycle count value, for comparator 0 only.

For address matching, the comparator can use a mask, so it can match a range of addresses.

On a successful match, the comparator generates one of the following:

- One or more DWT Data trace packets, containing one or more of:
 - The address of the instruction that caused a data access
 - An address offset, bits[15:0] of the data access address
 - The matched data value.

- A watchpoint debug event, on either the PC value or the accessed data address.
- A CMPMATCH[N] event that signals the match outside the DWT unit.

Breakpoint Unit 10

- The Breakpoint Unit (FPB) allows hardware breakpoints to be set.
 - It contains eight comparators which monitor the instruction fetch address and return a breakpoint instruction when a match is detected.
 - When the breakpoint instruction is executed, the processor halts in debug mode



The Cortex-M4 Breakpoint Unit (FPB) also supports Flash memory patching. This feature is intended for patching erroneous code by diverting execution to volatile memory at a given address.

- The Instrumentation Trace Macrocell (ITM) generates trace packets from four sources:
 - Software trace:
 - Software write to any of the 32 stimulus registers
 - Hardware trace packet from DWT
 - This may be a data trace event, a PC sample or a counter wrap-around
 - Local timestamp
 - A 21-bit counter in the ITM provides a timestamp for each trace packet, relative to the previous packet
 - Global timestamp
 - Timestamps can also be generated using the system-wide 64-bit timestamp from the TSGEN
- Trace packets are output on the ATB trace bus



Software can write directly to any of 32 x 32-bit Instrumentation Trace Macrocell (ITM) stimulus registers to generate packets. The permission level for each port can be programmed. When software writes to an enabled stimulus port, the ITM combines the identity of the port, the size of the write access, and the data written, into a packet that it writes to a FIFO. The ITM outputs packets from the FIFO onto the trace bus. Reading a stimulus port register returns the status of the stimulus register (empty or pending) in bit 0.

If multiple sources generate packets at the same time, the ITM arbitrates the order in which packets are output. The sources are listed here in descending order of priority.

The timestamp generator (TSGEN) provides a 64-bit common time base for all trace packet timestamps. This allows the trace analyzer to align traces coming from different sources according to the time at which the trace was generated. The local timestamps are not synchronized, and can run at

different frequencies, making it impossible to know the precise timing at which a trace was generated.

Note: The Cortex-M4 only uses the 48 LSBs of the global timestamp.

- The System Trace Macrocell (STM) generates trace packets from two sources:
 - **Software trace:**
 - A bus master (CPU, DMA, debugger) can write to any of the 65536 stimulus ports.
 - Each port contains 64 x 16-bit virtual registers. The register address determines the type of trace packet that will be generated.
 - Trace packet types may be data, flag or trigger, and may or may not include a timestamp.
 - **Hardware event trace:**
 - Any DMA request and/or acknowledge, and any interrupt assertion and/or de-assertion can be programmed to generate a trace packet
- Trace packets are output on the ATB trace bus



The System Trace Macrocell (STM) can be used for instrumentation of software. It is aimed primarily at the Cortex-A7, since the Cortex-M4 includes a simplified software trace unit, the ITM. Nevertheless, the STM is accessible to the Cortex-M4 core as well as the DMA and MDMA engines. It can even be used by the debugger.

Accesses to the stimulus ports can be « guaranteed » or « timing invariant ».

- Guaranteed accesses will always generate a trace packet, and will stall the access until the STM can accept it.
- Timing invariant accesses will always terminate immediately, so are less intrusive, but if the STM is not ready to accept the write due to its buffer being full, the data will be discarded and no packet generated.

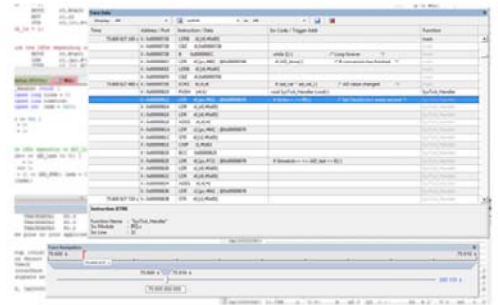
Trace packets always contain the identity of the master that generated them.

The STM control registers are accessible via the system debug

bus (APB-D).

Instruction Trace 13

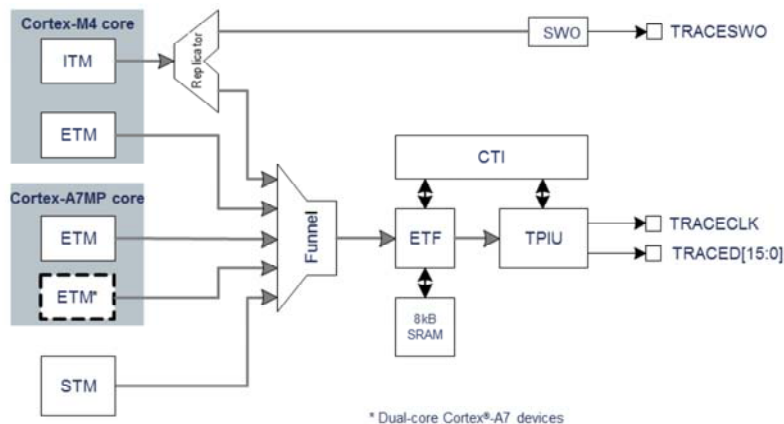
- The Embedded Trace Macrocell (ETM) generates trace packets which allow the execution of the software to be observed. The trace information includes:
 - The number of instructions executed in the same cycle
 - Changes in program flow
 - The current processor instruction state
 - The addresses of memory locations accessed by load and store instructions
 - The type, direction and size of a transfer
 - Condition code information
 - Exception information
 - Wait for interrupt state information
- Trace packets are output on the ATB trace bus



In the STM32MP1 series, the Embedded Trace Macrocell (ETM) is configured for instruction trace only; i.e. data accesses are not included in the trace information.

Trace Infrastructure 14

- Trace information from the ITM and ETM is routed via the Amba Trace Bus (ATB) to one or more of the trace sinks
 - The routing is performed by two ATB infrastructure components
 - Replicator – duplicates the trace packets on two ATB branches
 - Funnel – multiplexes several ATB branches onto one



The ITM, ETM and STM generate trace streams which are combined using the trace funnel. Some funnel parameters can be modified; for example, the number of bytes received on one input before switching to another – the less switching that occurs, the lower the overhead, but at the cost of increased latency. It is also possible to filter trace, for example the ITM trace can be removed from the TPIU (it can be output on the SWO instead).

Trace from the ITM (not the ETM) can be directed to the single wire trace port. In dual-core devices, the ITM trace from both cores can be directed to the SWO and combined in the SWO trace funnel. However since there is no formatting in the SWO, it is not possible for a trace port analyzer to separate the trace streams. Therefore it is recommended that the funnel be used to manually select one ITM at a time for output on the SWO. If both are needed the TPIU should be used.

- The trace packets are channeled to one of three destinations or “sinks”:
 - **Embedded Trace FIFO (ETF)**
 - This is an 8-Kbyte memory that can store trace packets in a circular buffer. The trace can be read out by software or by the debugger
 - **Trace Port Interface (TPIU)**
 - Trace packets are streamed out of the device via a 16-pin parallel port, accompanied by a synchronous clock signal. This requires connection of a trace port analyzer probe such as ULINKpro or DStream.
 - **Single Wire Trace port (SWO)**
 - ITM trace (from the Cortex®-M4) can be directed to the SWO and output using an asynchronous protocol (NRZ or Manchester). This can be read using the ST-Link or other adaptor and most commercial debugger tools.



The ETF can be used as a trace buffer for storing traces on-chip. The trace can be read by software, or by the debugger, or flushed via the trace port. If configured as a circular buffer, the trace will be stored continuously, so the most recent trace will overwrite the oldest. Alternatively, the FIFO full flag can be used to stop a trace when the buffer is full, and hence capture a trace at a particular point in time.

The ETF also acts (in hardware mode) to smooth the flow of trace to the TPIU. Since the trace stream tends to be bursty in nature, and the instantaneous bandwidth is much higher than that of the trace port, the buffer absorbs the peaks and regulates the flow to the trace port's maximum continuous bandwidth.

- TPIU parallel port
 - 1 to 16 data pins and a clock can be assigned to trace (GPIOs by default)
 - TRACECLK can operate at up to 133 MHz in DDR mode
 - 1Gbps maximum bandwidth (800 Mbps with ULINKpro)
- SWO single wire serial port (Cortex®-M4 only)
 - 1 asynchronous data pin multiplexed with JTDO
 - 100 Mbps bandwidth (Manchester encoded)



The trace port width can be programmed from 1 to 4 pins. The bandwidth scales proportionally to the number of pins and the TRACECLK frequency (selectable via a divider in the RCC). Full dual core instruction trace at maximum clock frequency is likely to require the maximum bandwidth. By applying filters and triggers to the trace sources (ETM notably), the average amount of trace data can be reduced, allowing a lower clock rate or reduced number of pins.

The TRACESWO pin is multiplexed with the JTDO signal, which is part of the JTAG interface. Hence single wire trace is only available when the serial wire debug (SWD) interface is enabled.

Cross Trigger Interface

17

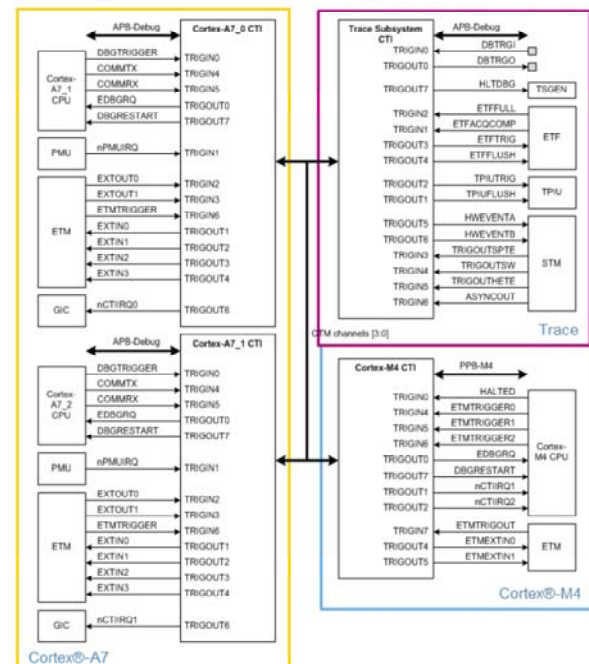
- The CTI propagates trigger events to other debug and trace components.

- Trigger event sources can include:

- Data watchpoints
- Hardware breakpoints
- Profiling counter events
- Trace buffer full/empty
- External trigger signal
- Processor halt/restart

- Trigger events at the destination can cause:

- Trace start/stop
- Trace buffer flushing
- Processor halt/restart
- External trigger signal output
- Processor interrupt



Cross triggering can be used in multi-core devices to halt both cores simultaneously. When one core hits a breakpoint, its “halted” output (indicating it has entered debug mode) propagates to the other core and causes it to enter debug mode as well. Similarly, both cores can restart simultaneously. The cross-trigger feature can also be used to halt the processor with an external trigger signal (this might be an edge on one of the IO pins).

There is a Cross-Trigger Interface (CTI) for each of the processor cores, as well as a system CTI connected to the trace components (ETF, TPIU, STM) and external trigger signals.

To use any of the cross-trigger features, the CTIs must be programmed accordingly by the debugger. The required trigger input signals (TRIGINn) and trigger output signals (TRIGOUTn) need to be connected to the cross-trigger matrix (CTM). The CTM comprises up to four channels allowing four different events to be propagated in parallel. Trigger inputs can be combined in the CTI so that any one of the combined inputs will cause an event on the

connected channel. Similarly, a channel can be connected to several trigger outputs, so that one event can trigger multiple actions.

- The “MCU debug” block enables device-specific debug features
 - Device identity
 - Standard location for the reading the device identity code register
 - Emulation of low power modes
 - Maintains the power and clock when the device enters a low power mode (SLEEP, STOP, STANDBY) so that debug access is still possible
 - Peripheral clock “freeze” in debug mode
 - Freezes the RTC, TIM, LPTIM and watchdog (IWDG, WWDG) timer counters, as well as the SMBUS and FDCAN timeout counters, while the processor is halted
 - Domain debug clock enable
 - Disables the clocks to debug devices in each power domain when not required, to save power
 - External trigger direction
 - Controls the direction (input/output) of the bi-directional TRGIO external trigger pin



The DBGMCU is located on the debug APB bus and can be accessed by the debugger via the APB access port AP2. It is also accessible by the processor(s) in the debug APB address space. The DBGMCU_IDC register provides the device ID and revision codes in STM32 standard format. The information is also available in the debug port (DP_TARGETID register – accessible only to an external debugger) and in the system debug ROM table registers (SYSROM_PIDR[2:0] – accessible also by software).

Low power mode emulation means that the debugger connection is not lost when entering low power mode. It eliminates the need to replace the low power entry command (for example, WFI/WFE) by a while() loop. On exit, the device is in the same state as if the emulation was not active (apart from any changes made by the debugger during the low power mode emulation).

Peripheral clock freeze is particularly useful to prevent a watchdog timeout from resetting the device while debugging, without having to re-arm the watchdog with the debugger. It also allows

timer values to be inspected and corresponding interrupts to be suspended until “normal” operation is resumed.

The debug clock enable bits ensure that the debug blocks are only clocked when needed. This avoids unnecessary power consumption, since apart from the DAP, all blocks are clocked with the ungated domain clock.

On certain packages, the TRGIN and TRGOUT pins are not available, only the bi-directional pin is used, and the direction must be chosen using the TRGOEN bit.